

Executive Roadmap Overview

- Month 1: Core Engine & Ingestion (Foundation)
- Month 2: Automated Communication & Tokenized Guest PWA
- Month 3: Logistics, Geospatial Discovery & Ledger
- Month 4: Security Hardening, Multi-Tenancy & E2E Testing
- Month 5: Beta Launch & Private Alpha Event Executions
- Month 6: Full SaaS Production Launch & Scaling

Phase 1: Month 1 — Core Backend, Database & AI Ingestion Engine

Primary Objective: Build the modular monolith backend foundation, establish multi-tenant PostgreSQL schemas, and implement the structured AI parsing pipeline.

Sprints 1 & 2 (Days 1–30)

- **Infrastructure & CI/CD Setup:**
 - Set up repository structure (TypeScript / Node.js / Express or FastAPI).
 - Initialize PostgreSQL database with Prisma ORM and execute multi-tenant schema migrations.
 - Set up Dockerized local environments and staging CI/CD pipelines (GitHub Actions $\rightarrow$ AWS EKS/ECS).
- **Module 1 Engineering (AI Ingestion):**
 - Build `POST /ingestion/parse` endpoint integrating LLM function calling with strict JSON output schemas.
 - Develop parser fallback logic to handle unstructured text dumps, spreadsheets, and voice-to-text transcripts.
 - Write unit tests verifying parsing accuracy ($>92\%$ target) across edge-case guest lists and schedules.
- **Key Milestone:** Raw unstructured text converts reliably into typed PostgreSQL records (TenantEvent, Guest, SubEvent).

Phase 2: Month 2 — WhatsApp Automation & Zero-Auth Guest PWA

Primary Objective: Build the Meta WhatsApp Cloud API integration, establish the interactive RSVP webhook handler, and launch the tokenized Guest PWA interface.

Sprints 3 & 4 (Days 31–60)

- **Module 2 Engineering (WhatsApp & RSVPs):**
 - Integrate Meta WhatsApp Business API SDK for outbound template delivery.
 - Build `POST /webhooks/whatsapp` receiver handling interactive button taps (`[Attending]`/`[Declining]`).
 - Implement Redis task queues (BullMQ/RabbitMQ) for asynchronous processing of inbound RSVP webhooks within $<200\text{ms}$.
- **Module 5 Engineering (Guest PWA Core & Tokens):**
 - Implement `TokenService` using HMAC-SHA256 cryptographically signed query strings for zero-auth access.
 - Build mobile-first Progressive Web App (Next.js/React + Tailwind) rendering personalized travel info and schedules.
 - Implement S3 presigned URL generation (`POST /media/presigned-url`) for browser-based direct photo uploads with client-side compression.
- **Key Milestone:** Host triggers a WhatsApp blast; guest taps a button to RSVP, instantly receiving a tokenized PWA link to view their itinerary and upload photos.

Phase 3: Month 3 — Multi-Tenant Ledger, Geospatial Discovery & Logistics

Primary Objective: Complete the Host Admin Portal, integrate Google Places API for vendor sourcing, build the multi-tenant financial ledger, and connect fleet dispatch logic.

Sprints 5 & 6 (Days 61–90)

- **Module 3 & 4 Engineering (Vendor Search & Bidding):**
 - Build `POST /vendors/discover` integrating Google Places API (New) with spatial radius filtering.
 - Develop automated vendor outreach messaging templates to request availability and pricing quotes via text.
- **Module 3 Engineering (Multi-Tenant Financial Ledger):**
 - Implement double-entry accounting schema allocating funds across multiple family contributor pools.
 - Connect approved vendor quotes directly to committed liabilities in the ledger.
- **Host Admin Portal Development:**
 - Build Host Management Dashboard for real-time guest list filtering, budget monitoring, and timeline adjustments.
- **Key Milestone:** Host can manage multi-funder budgets, discover local vendors on a map, and auto-assign confirmed guests to fleet shuttles.

Phase 4: Month 4 — Security Hardening, RLS & E2E System Integration

Primary Objective: Enforce strict PostgreSQL Row-Level Security (RLS), perform security/penetration testing, and execute end-to-end load tests.

Sprints 7 & 8 (Days 91–120)

- **Multi-Tenancy & Security Implementation:**
 - Enable and test PostgreSQL Row-Level Security (RLS) policies across all tables (`app.current_event_id`).
 - Implement API rate-limiting per `eventId` and audit data privacy compliance against DPDP Act/GDPR frameworks.
 - Conduct internal penetration testing focusing on cross-tenant data isolation and token tampering.
- **Real-Time Subscriptions & Optimization:**
 - Implement WebSocket server (`wss://api.shagun.app/v1/realtime`) using Redis Pub/Sub for real-time timeline shifts and live photo wall feeds.
 - Optimize PWA performance to achieve $<1.5\text{s}$ First Contentful Paint (FCP) on mobile 4G networks.
- **Key Milestone:** Platform achieves 100% tenant isolation, zero cross-tenant leakages, and passes simulated load tests of 1,000 concurrent guests.

Phase 5: Month 5 — Private Alpha / Beta Event Executions & Feedback Loop

Primary Objective: Execute 2–3 live Alpha events with real hosts and guests, monitor real-time telemetry, and patch operational bottlenecks.

Sprints 9 & 10 (Days 121–150)

- **Controlled Live Deployment:**
 - Deploy v1.0 Release Candidate to production infrastructure (AWS/Cloudflare).
 - Run 2–3 real-world events as managed pilots with high-touch support from the engineering team.
- **Telemetry & Observability Monitoring:**
 - Monitor WhatsApp delivery rates, webhook latency, S3 presigned upload success rates, and WebSocket connection dropoffs using Datadog/Sentry.
 - Gather host and guest UI/UX feedback to identify points of friction.
- **Patch & Refinement Sprint:**
 - Refine LLM parser prompts based on edge-case text inputs encountered during live onboarding.
 - Optimize mobile PWA camera bridge performance across low-end devices.
- **Key Milestone:** Successfully execute 3 live events with >75% guest RSVP conversion and zero backend downtime.

Phase 6: Month 6 — Full SaaS Production Launch & Commercial Scaling

Primary Objective: Open public self-serve onboarding, launch monetization tiers, and scale customer acquisition channels.

Sprints 11 & 12 (Days 151–180)

- **Public SaaS Onboarding & Billing Setup:**
 - Integrate Stripe/Razorpay subscription and usage-based billing gateways for host SaaS licenses and WhatsApp message markups.
 - Launch automated self-serve host onboarding flow (shagun.app/signup).
- **Go-To-Market (GTM) Execution:**
 - Roll out marketing campaigns targeting event planners, wedding planners, and host families.
 - Establish vendor partnership portal for featured local vendor listings.
- **Infrastructure Auto-Scaling:**
 - Configure Kubernetes Horizontal Pod Autoscalers (HPA) and Aurora PostgreSQL serverless auto-scaling to handle variable weekend usage spikes.
- **Key Milestone:** Public platform launch with active self-serve host signups, automated payment processing, and multi-event concurrent scaling.

6-Month Resource Allocation & Deliverables Matrix

Month	Engineering Focus	Primary Deliverable	Risk Mitigation Strategy
Month 1	Backend Core & AI Parser	Working Ingestion API & Database Schemas	Enforce strict JSON output schemas on LLM outputs to prevent schema corruption.
Month 2	Meta Webhooks & Guest PWA	WhatsApp Interactive RSVPs & Tokenized PWA	Process webhooks asynchronously using Redis queues to prevent Meta timeouts.
Month 3	Ledger, Discovery & Host UI	Full Host Admin Dashboard & Google Places API	Cache Google Places queries heavily to prevent API budget overruns.
Month 4	Security & RLS Hardening	Verified Multi-Tenant Security & Real-Time WSS	Enforce database-level RLS policies to guarantee zero cross-tenant leaks.
Month 5	Live Pilots & Beta Testing	3 Successfully Executed Real-World Events	Keep engineering team on standby during live pilot events for instant hotfixes.
Month 6	Public Launch & Scale	Self-Serve Onboarding, Billing & Auto-Scaling	Implement horizontal pod autoscaling to handle unpredictable event traffic.